

Translated

From: e-maxx.ru

Finding bridges in a graph in $O(N+M)$

We are given an undirected graph. A bridge is defined as an edge which, when removed, makes the graph disconnected (or more precisely, increases the number of connected components in the graph). The task is to find all bridges in the given graph.

Informally, the problem is formulated as follows: given a map of cities connected with roads, find all "important" roads, i.e. roads which, when removed, cause disappearance of a path between some pair of cities.

The algorithm described here is based on [depth first search](#) and has $O(N+M)$ complexity, where N is the number of vertices and M is the number of edges in the graph.

Note that there is also the article [Finding Bridges Online](#) - unlike the offline algorithm described here, the online algorithm is able to maintain the list of all bridges in a changing graph (assuming that the only type of change is addition of new edges).

Algorithm

Pick an arbitrary vertex of the graph $root$ and run [depth first search](#) from it. Note the following fact (which is easy to prove):

- Let's say we are in the DFS, looking through the edges starting from vertex v . The current edge (v, to) is a bridge if and only if none of the vertices to and its descendants in the DFS traversal tree has a back-edge to vertex v or any of its ancestors. Indeed, this condition means that there is no other way from v to to except for edge (v, to) .

Now we have to learn to check this fact for each vertex efficiently. We'll use "time of entry into node" computed by the depth first search.

So, let $tin[v]$ denote entry time for node v . We introduce an array low which will let us store the earliest entry time of the node found in the DFS search that a node v can reach with a single edge from itself or its descendants. $low[v]$ is the minimum of $tin[v]$, the entry times $tin[p]$ for each node p that is connected to node v via a back-edge (v, p) and the values of $low[to]$ for each vertex to which is a direct descendant of v in the DFS tree: